

- `<a v-bind:href="url"> ... </a>`
-
- `<!-- shorthand -->`
- `<a :href="url"> ... </a>`
-
- `<!-- shorthand with dynamic argument (2.6.0+) -->`
- `<a :[key]="url"> ... </a>`

Normal HTML may not look similar to this. But, the `@` and `:` can be perfectly parsed by all Vue-supported browsers, and thus, they are valid characters that go well for the attribute names. Furthermore, there is no trace of these characters in the final rendered markup. This is not an optional shorthand syntax. Nevertheless, you would like to make maximum usage of this template when you learn more about it.

Conclusion

Vue.js was at first developed for prototyping. It is one of the most amazing tools to handle graphics, interactive elements, and a lot of animations with proper data-binding. On the other hand, it only requires JS, CSS, and HTML. So, there are no acute things specific to Vue. With JavaScript, it can be easily molded into any other project that involves JS. It is compatible with several frameworks and technologies such as Django, Rails, Express, and Laravel.

The components in Vue are identified as encapsulated elements for the interface. You can write these in JavaScript, CSS, and HTML without distributing them into separate files. Such splitting of files can be best used for unit-testing, code readability, and component reusability.

Being a leading framework in the industry for more than 5 years, it has gained a lot of prominence for its compatibility with various other sets of tools. Its adoption is easy to understand. With Vue, you do not need an in-depth knowledge of TypeScript, JSX, or libraries. All you need is to know JavaScript, CSS, and HTML. The framework is featherweight. Thus, it does not take much time to download and install the library. This greatly influences the UX and SEO of your webpage. **d**